

Figure 6: SQuAD performance vs input sequence length. The compression ratio is fixed at 1/4. Uses Vicuna 1.5 7B with 16k maximum sequence length against our SQuAD (train) task with 7 (default) to 63 confusion contexts to increase the sequence length. We believe the drop in performance at 3k tokens is an artefact of the RoPE scaling and fine-tuning procedure used to extend the context window of the Vicuna model.

billion parameters. Results from the largest models are presented in Table 2, with further results in Figures A1 to A3.

We observe that SparQ Attention performance is robust across all tasks and models tested, as compression ratios of 1/2 to 1/8 are readily achievable with little to no loss in task performance. H₂O can attain good performance on some tasks such as TriviaQA and WikiTest-103, although other tasks, including SQuAD and Text Repetition, are more challenging and notable degradation occurs. LM-Infinite performance degrades across all tasks, demonstrating that the tasks do not permit the trivial solution of discarding the long input sequence.

5.3. Sequence Length Scaling

The sequence lengths of different examples in our main tasks vary between 1k and 2k tokens, whereas many LLMs support sequence lengths far greater than this. We considered two tasks to evaluate how SparQ Attention performs as sequence length scales.

The first task is a variation of SQuAD, which increases both sequence length and task difficulty by increasing the number of confusion contexts present in the prompt, which is akin to increasing the number of retrieved documents with a retrieval augmented generation system (Borgeaud et al., 2022). We test SparQ Attention and H₂O in this setting using Vicuna (Chiang et al., 2023), a descendent of Llama 2 that has been adapted for longer sequences. Both SparQ Attention and H₂O are configured to maintain a fixed compression ratio versus the dense baseline (keeping $r = 32$ and modifying k to maintain 1/4 compression). The results in Figure 6 show that SparQ Attention is scalable to large sequences, as it can maintain performance up to 128k sequence length.

Table 3: Needle in a haystack results, averaged over all depths and sequence lengths S within the specified range. See Figure A4 for full heatmap results and more details.

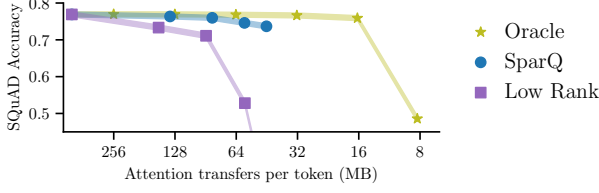
S Range	Compression	LM- ∞	H ₂ O	SparQ
8k - 16k	1	100%	100%	100%
	1/4	23.5%	5.9%	100%
	1/8	11.8%	2.9%	79.4%
16k - 24k	1	100%	100%	100%
	1/4	23.5%	8.8%	100%
	1/8	11.8%	5.9%	100%
24k - 32k	1	90.4%	90.4%	90.4%
	1/4	23.5%	10.3%	90.4%
	1/8	11.8%	5.9%	87.5%

The second task evaluated is *needle in a haystack*, in which a “text needle” is inserted into the context at a certain depth, and the model is tasked with retrieving information from the needle. The exact implementation of this task we used is outlined in Dhinakaran (2024). We compare SparQ Attention with H₂O and LM-Infinite over a range of different compression ratios. The results, as seen in Table 3 and Figure A4, show that SparQ Attention achieves performance very close to the dense attention baseline, even in high-sparsity settings.

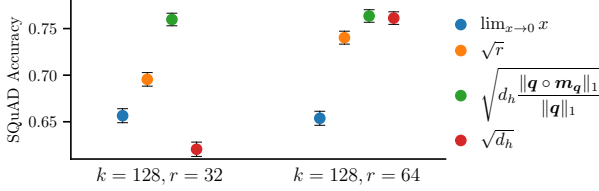
5.4. Ablations

Key cache compression The first step in SparQ Attention involves reading r components of the key cache to approximately determine which keys yield the highest attention scores. To examine the practical trade-off of the approximation, we look at how SparQ Attention performs when compared to a theoretical upper-bounding “oracle” which provides the exact top- k keys without requiring any data transfer to calculate the top- k . The results in Figure 7a show that SparQ Attention retains comparable performance to the oracle for a wide range of compression ratios, and attains considerably higher performance than a baseline compression scheme, in which a random low rank projection of K is transferred from memory.

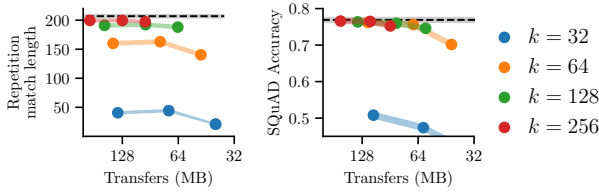
Approximate softmax temperature To empirically support our statistical analysis of α agreement shown in Figure 4f, we evaluate a number of different viable temperature settings, including the square root of the head dimension ($\tau = \sqrt{d_h}$), the square root of the rank ($\tau = \sqrt{r}$), and the temperature proposed in Equation (9). We also consider the scenario where we do not reallocate mass to mean value ($\alpha = 0$), which corresponds to the limit of the temperature tending towards 0. We find that our proposed temperature performs best, as shown in Figure 7b.



(a) Accuracy results of SparQ Attention and a random low rank compression scheme against an oracle top- k selector.



(b) Comparison of different softmax temperatures for approximate attention scores for two different hyperparameter configurations.



(c) Results for Repetition and SQuAD tasks with $r \in \{16, 32, 64\}$.

Figure 7: Ablations results, investigated on Llama 2 7B.

Hyperparameter selection The reduction of data transfer attained by SparQ Attention is controlled by its two hyperparameters, k and r . Reducing either of these variables will improve the bandwidth efficiency, but can negatively impact task performance. Figure 7c shows the relationship between k and r on both of these factors. Based on these results, we propose a simple recipe of setting $k = 128$ and tuning r to maintain a good trade-off between data transfer and task performance for a range of models and tasks.

6. Benchmarking

The results above use a theoretical cost model of total memory transfers (the number of scalar elements transferred to and from memory per token), allowing us to evaluate SparQ Attention independently of the specific hardware setup and number formats used. To validate our findings, we performed a set of microbenchmarks of an attention operation in isolation, in addition to end-to-end performance benchmarks.

SparQ Attention benefits from two optimisations. The first is to store K twice, in both d_h -contiguous and S -contiguous layouts, since this allows for an efficient gather (indexing)

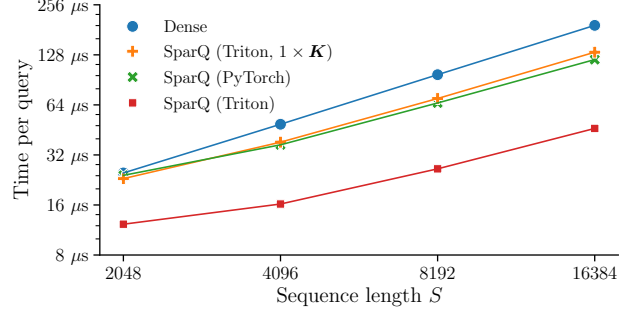


Figure 8: Microbenchmark results for batch size 64, 32 heads, $d_h = 128$, $r = 32$, $k = 128$, A100 (40GB).

Table 4: GPU microbenchmark performance with batch size 64, sequence length $S = 4096$, $r = 32$, $k = 128$. The theoretical speed-up is $6.4\times$.

Kernel	A100 (40GB)	A10G
Dense	49 μ s ($1\times$)	128 μ s ($1\times$)
SparQ (Triton, $1\times K$)	38 μ s ($1.28\times$)	79 μ s ($1.63\times$)
SparQ (PyTorch)	37 μ s ($1.33\times$)	78 μ s ($1.63\times$)
SparQ (Triton)	16 μs ($3.02\times$)	31 μs ($4.17\times$)

on either axis, at the cost of 50% extra memory usage. The second optimisation is to use a fused gather-then-matmul operation to avoid writing the result of the gather to memory.

6.1. Microbenchmarks

We tested multiple implementations of baseline and SparQ Attention on IPU using the Poplar C++ interface and GPU using PyTorch (Paszke et al., 2019). In all cases, we used the Llama 2 7B shape parameters: 32 heads, $d_h = 128$. The implementations tested were: **Dense** baseline, choosing the faster of a plain PyTorch implementation and the builtin `scaled_dot_product_attention`, **SparQ (Triton)**, storing K twice and using fused gather-then-matmul kernels written using Triton (Tillet et al., 2019), **SparQ (PyTorch)**, with no Triton and **SparQ (Triton, $1\times K$)**, storing K in d_h -contiguous layout only, for no additional memory cost. In an example configuration running on a single IPU from a Bow Pod₁₆, batch size 1, sequence length $S = 16384$, the dense baseline achieves 40.4 ms/query, while SparQ ($r = 32$, $k = 128$) achieves 5.28 ms/query for a speedup of $7.41\times$ (the theoretical speedup of SparQ is $7.53\times$). This near-perfect speedup is achieved because attention is strongly memory bound when using remote memory. In contrast, the baseline running in local SRAM takes 134 μ s for a $345\times$ speedup, but this is only practically achievable when the whole model fits in SRAM.

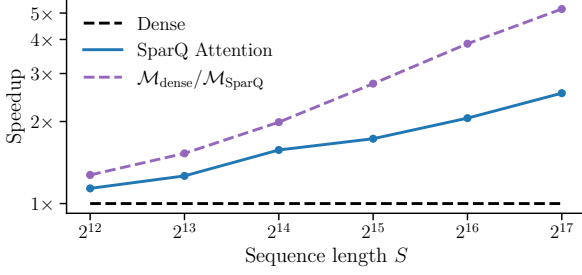


Figure 9: End-to-end CPU speedup results for SparQ Attention compared to the dense baseline with Llama 2 7B. This was achieved with batch size 1 and a compression ratio of 1/8, with model weights represented in 8-bits and the KV cache represented in 16-bits. The $\mathcal{M}_{\text{dense}}/\mathcal{M}_{\text{SparQ}}$ line is a theoretical result, illustrating the upper bound of speedups attained with SparQ Attention.

Our achieved GPU speed-ups are presented in Table 4, and the performance trend with sequence length is shown in Figure 8. Standard error for all results given is $< 1\%$ of the mean. See Appendix F for further details.

These microbenchmark results show that the theoretical benefits of SparQ Attention can yield substantial wall-clock time speedups on current hardware. Further work is needed to show improvements for small batch sizes, and to investigate alternatives to storing $\mathbf{K}$ twice.

6.2. End-to-End Performance

In addition to the positive microbenchmark results, we further highlight the practical improvements SparQ Attention offers by benchmarking performance of the entire Transformer model on both CPU and GPU, implemented in `llama.cpp` and `gpt-fast` (Meta, 2023) respectively. In both cases, we measured the time it took to generate a single token, given an existing sequence length S .

CPU benchmarking We evaluated CPU benchmarking performance on AMD EPYC systems with up to 256GB memory. The results, as seen in Figure 9, show that SparQ Attention attains speedups at all sequence lengths evaluated, compared to the dense baseline. At the longest sequence lengths considered, SparQ Attention achieves a $2.5\times$ speedup, showcasing the benefits of reducing the data transfer associated with attention.

GPU benchmarking Our end-to-end GPU implementation is evaluated on a single H100 PCIe with 80GB memory. Despite utilising high-bandwidth memory, GPU inference still achieves end-to-end speedups when using SparQ Attention on modest sequence lengths, as seen in Figure 10.

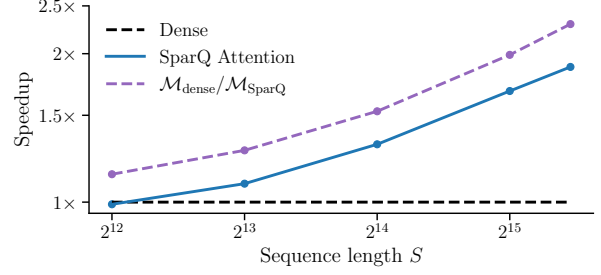


Figure 10: End-to-end H100 speedup results for SparQ Attention compared to the dense baseline with Llama 2 7B. These speedups were achieved with batch size 1 and a compression ratio of 1/8, with both model weights and KV cache represented in 16 bits. The $\mathcal{M}_{\text{dense}}/\mathcal{M}_{\text{SparQ}}$ illustrates the theoretical maximum performance attainable under this setting.

7. Related Work

Efficient attention methods have been a very active area of research (Tay et al., 2020b). Schemes such as *Sparse Transformers* (Child et al., 2019), *Combiner* (Ren et al., 2021), *Longformer* (Beltagy et al., 2020), *BigBird* (Zaheer et al., 2020), *Reformer* (Kitaev et al., 2020) and *Sparse Sinkhorn Attention* (Tay et al., 2020a) have been developed to increase efficiency of the attention mechanism by extracting information from the most salient tokens in the sequence or approximating dense attention maps. Two schemes that reduce memory footprint and data transfer of the attention operation, while maintaining quadratic complexity are *Multi-Query Attention* (MQA) (Shazeer, 2019) and *Grouped-Query Attention* (GQA) (Ainslie et al., 2023) that share each KV head across multiple query heads. These methods form part of the architecture: they must be implemented during pre-training, carry varying task performance trade-offs, and may affect model quality and stability.

An emerging area of research similar to SparQ Attention aims to only adapt the inference procedure of a pre-trained model. The simplest method of this category is part of *FlexGen* (Sheng et al., 2023), and calculates exact attention scores, retrieving only the values associated with the top- k scores. This process uses the full key cache to produce attention scores, limiting the asymptotic reduction of the memory transfers to only 50%. LM-Infinite (Han et al., 2023) and StreamingLLM (Xiao et al., 2023) employ a fixed sparsity pattern preserving the most recent tokens and a few initial tokens for better attention efficiency, but are not selective in their cache lookup.

Eviction schemes cache only a subset of keys and values, by continually deleting tokens that are uninformative for future outputs. By reducing the cache size itself, both the